Christopher Buenrostro
Josh Goldberg
Anthony Martinez

Arcade Rewards Banking
CST 363 Project

Our project proposal is to design a database for an arcade rewards, banking, and redemption system. People play games at an arcade to win digital tickets, which are stored in an account for them, until they go to the redemption counter and can spend them on prizes.

Problem Statement

Database Design Scenario — Arcade Management System

This database design will support the operations of an arcade where customers use reloadable cards to purchase tokens, play machines, earn tickets, and redeem prizes.

Each customer is registered in the system with a first and last name. Every customer may own one or more arcade cards, but each card belongs to exactly one customer. Each card has a status (either active or inactive), a token balance, and a ticket balance. Token and ticket balances cannot be negative.

Customers add tokens to their cards through token purchases. Each purchase must record the card used, the amount of tokens added, and the date and time it occurred. A card may have many token purchases over time. Only an active card may be used.

The arcade contains multiple machines. Each machine has a unique machine name and a machine type (for example, racing, claw, basketball, rhythm, etc.). A machine can be played many times by many different customers.

When a customer plays a machine, a play session is recorded. For each play session, we must record the machine played, the card used, the score achieved, and the date and time the game was played. A score cannot be negative. A single card may have many play sessions, even on the same machine and on the same day.

Customers spend tokens on game plays. A play session must be paid for. We must record the date and time of a token spent. A payment is reflected as a deduction from the card's total token balance, and a payment deduction must not result in a balance falling below zero.

Some machines award tickets based on a player's score. Each machine may have multiple ticket rules that define a minimum score, a maximum score, and the number of tickets awarded

for scores within that range. Score ranges must be valid (the minimum score cannot exceed the maximum score), and ticket awards cannot be negative. Ranges should not overlap. If a machine's rules need adjustment, new rule records should be made. Only one machine rule should actively cover a score range.

When a play session earns tickets, a ticket award record is created. For each ticket award, we must record the play session, the rule that was applied, and the date and time the tickets were granted. Each play session may result in at most one ticket award.

Customers may redeem prizes using the tickets stored on their card. A redemption event must record the card used and the date and time of redemption. A single redemption may include multiple prizes.

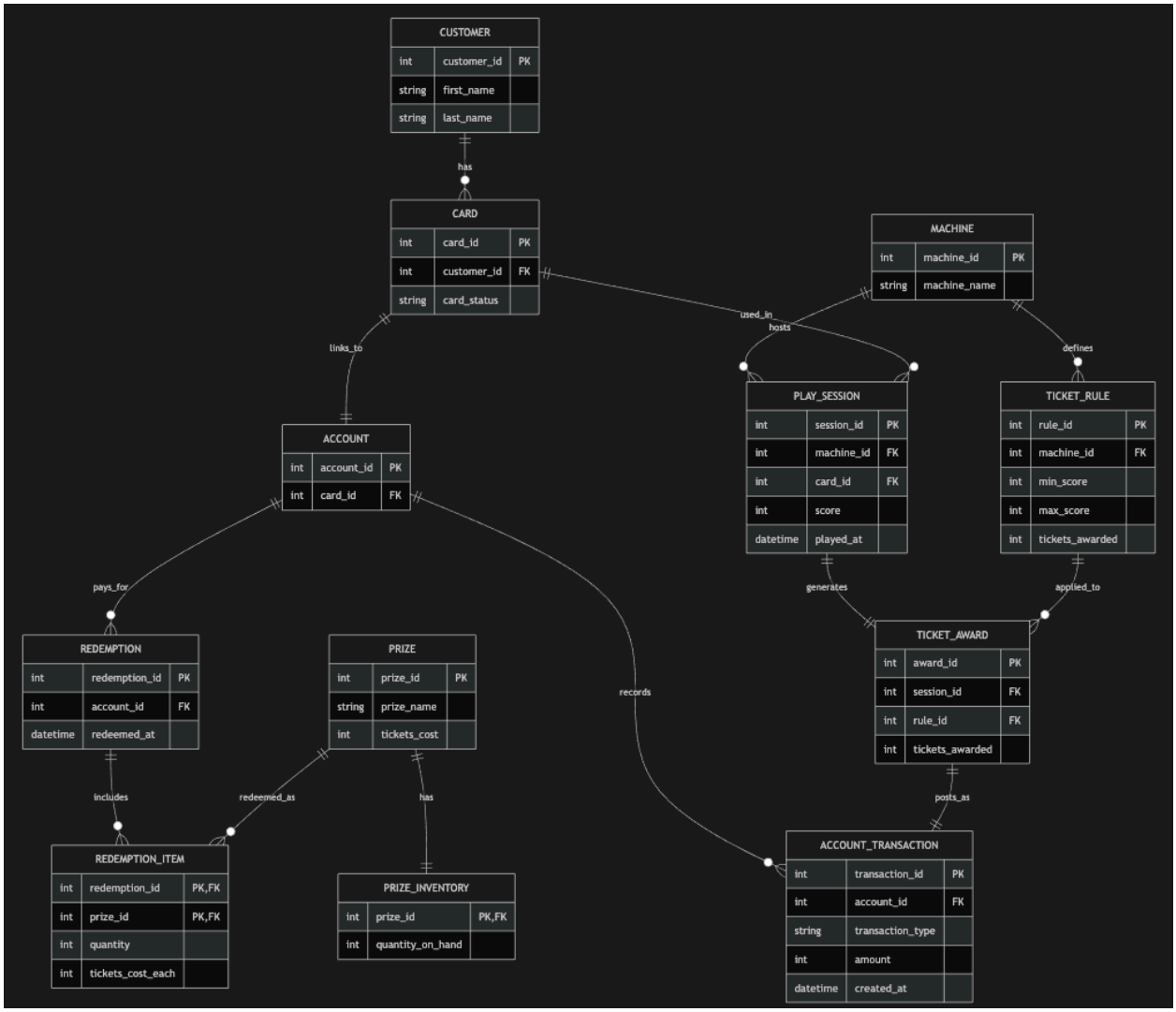
Each prize has a unique name and a standard ticket cost, which must be greater than zero. Because prize costs may change over time, each redemption item must also record the ticket cost per item at the time of redemption. For each redemption item, we must record the prize, the quantity redeemed, and the cost per item. A quantity must be greater than zero.

The system must also track prize inventory. For each prize, we must record the quantity currently on hand. Inventory quantities cannot be negative. Inventory is tracked per prize, not per redemption.

For example, a customer could purchase 100 tokens onto their card, use 10 tokens to play a basketball machine, score 750 points, earn 25 tickets based on the applicable score rule, and later redeem 50 tickets for two small plush prizes in a single redemption transaction. All of these activities must be recorded accurately and linked together through the appropriate relationships.

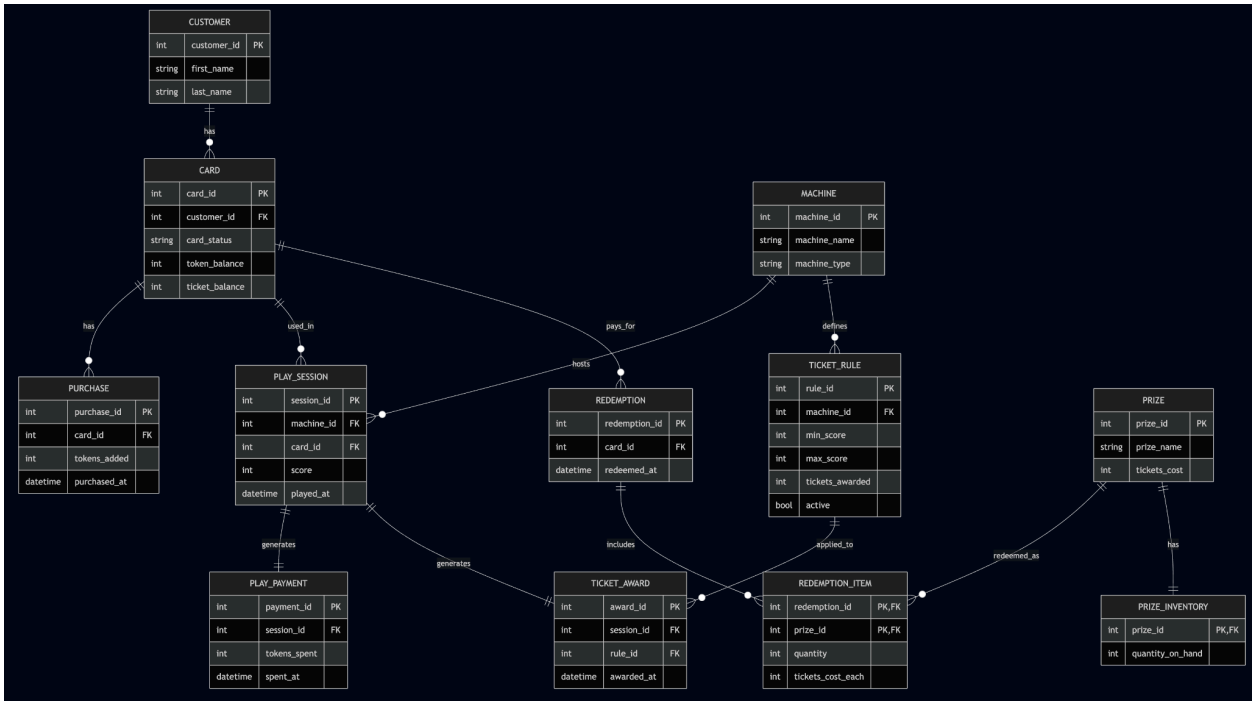
Preliminary Design

Our preliminary design separated CARD and ACCOUNT as distinct entities, where ACCOUNT functioned as the wallet that stored ticket transactions. Ticket earning and redemption were modeled using an ACCOUNT_TRANSACTION ledger. Redemption was tied directly to ACCOUNT rather than CARD. At this stage, the design focused on capturing transactional history through ledger tables, but it introduced additional complexity. As the design evolved, we simplified the model by removing the ACCOUNT entity and consolidating wallet functionality into CARD. This reduced redundancy and clarified relationships while maintaining third normal form (3NF).



Final 3NF Design

The final schema is in Third Normal Form (3NF) because each table represents a single entity, every non-key attribute depends only on the primary key, and no transitive dependencies remain. Derived or redundant attributes were removed during refinement, such as eliminating tickets_awarded from TICKET_AWARD and instead referencing immutable TICKET_RULE records. Composite keys are used appropriately in REDEMTION_ITEM to model the many-to-many relationship between redemptions and prizes. Balance values (token_balance, ticket_balance) are stored on CARD as snapshot attributes, while increases and decreases are recorded through event tables (PURCHASE, PLAY_PAYMENT, TICKET_AWARD, and REDEMPTION_ITEM). Foreign key constraints clearly define relationships between entities and enforce referential integrity. Together, these design choices ensure the schema avoids redundancy, maintains data integrity, and satisfies 3NF requirements.



Mermaid code for ER diagram

```
erDiagram
  CUSTOMER ||--o{ CARD : has

  CARD ||--o{ PURCHASE : has
  PLAY_SESSION ||--|| PLAY_PAYMENT : generates

  MACHINE ||--o{ PLAY_SESSION : hosts
  MACHINE ||--o{ TICKET_RULE : defines
  CARD ||--o{ PLAY_SESSION : used_in
```

```
PLAY_SESSION ||--|| TICKET_AWARD : generates
TICKET_RULE ||--o{ TICKET_AWARD : applied_to

CARD ||--o{ REDEMPTION : pays_for
REDEMPTION ||--o{ REDEMPTION_ITEM : includes
PRIZE ||--o{ REDEMPTION_ITEM : redeemed_as
PRIZE ||--|| PRIZE_INVENTORY : has


CUSTOMER {
  int customer_id PK
  string first_name
  string last_name
}

CARD {
  int card_id PK
  int customer_id FK
  string card_status
  int token_balance
  int ticket_balance
}

PURCHASE {
  int purchase_id PK
  int card_id FK
  int tokens_added
  datetime purchased_at
}

PLAY_PAYMENT {
  int payment_id PK
  int session_id FK
  int tokens_spent
  datetime spent_at
}

MACHINE {
  int machine_id PK
  string machine_name
  string machine_type
}

PLAY_SESSION {
```

```
  int session_id PK
  int machine_id FK
  int card_id FK
  int score
  datetime played_at
}

TICKET_RULE {
  int rule_id PK
  int machine_id FK
  int min_score
  int max_score
  int tickets_awarded
  bool active
}

TICKET_AWARD {
  int award_id PK
  int session_id FK
  int rule_id FK
  datetime awarded_at
}

REDEMPTION {
  int redemption_id PK
  int card_id FK
  datetime redeemed_at
}

REDEMPTION_ITEM {
  int redemption_id PK, FK
  int prize_id PK, FK
  int quantity
  int tickets_cost_each
}

PRIZE {
  int prize_id PK
  string prize_name
  int tickets_cost
}

PRIZE_INVENTORY {
  int prize_id PK, FK
  int quantity_on_hand
```

```
}
```

Solution Description

In the card table, token_balance and ticket_balance attributes describe how many tokens and tickets, respectively, are currently available for spending by the customer with the card. There are constraints to ensure that the balance attributes are non-negative, and that the card_status attribute is only either ‘active’ or ‘inactive’. These must be updated in a transaction when they are modified. Token_balance is incremented with inserts to purchase and decremented with inserts to play_payment. Ticket_balance is incremented with inserts to ticket_award and decremented with inserts to redemption. The total amount of tickets redeemed is the sum of redemption_item(quantity*tickets_cost_each) for a redemption association.

Ticket rules have a constraint that needs to be enforced in the application. No two active ranges for a machine can overlap. This can be checked during a transaction that sets and unsets the active flag in rules for a machine appropriately to maintain non-overlapping ranges.

When rules need adjustment, new records are inserted rather than modifying existing ones. This preserves the history of rules and awards that were won in the past and maintains relational integrity with the ticket_awards. Sessions that have no matching rules for a final score do not earn any ticket awards. The 1:1 relationship between play_session and ticket_award is enforced by a foreign key and unique constraint.

Each play_sessions has one and only one play_payment, also enforced with a foreign key and unique constraint.

The redemption_items table is a composite key of redemption_id and prize_id so each prize only appears once per redemption, but the quantity of the prize may be greater than one to support a redemption of multiple instances of the same prize. The ticket_cost_each column captures a point-in-time cost of the prize so if a prize’s cost is later adjusted the history of the redemption ticket amount doesn’t change.

Prize inventory is tracked separately rather than being derived from the redemption history. It must be decremented by the quantity of each redemption in a transaction, and is constrained to non-negative amounts so the redemption will fail if the inventory level would otherwise fall below zero. Using a transaction also ensures that any single prize in inventory is only granted to a single customer.

The schema is designed to support real world scalability by separating heavy transactional data from static data. Tables such as play_session, play_payment, ticket_award, purchase, and redemption_time are expected to grow rapidly over time, while tables such as customer,

machine, ticket_rule, and prize change infrequently. This separation allows indexing storage optimization.

From an auditing perspective every financial and gameplay action is completely traceable. Token purchases, token spending, ticket awards, and prize redemptions are never overwritten or deleted during normal operation. Instead each action generates a timestamped record that can be reviewed at a later time. This supports dispute resolution such as a customer claiming to not receive ticket or card funding, fraud detection and operational reporting. Our historical ticket rules/data preserved and not modified allows verifiable awards even if scoring policies change.

The design establishes a balance between performance efficiency with accountability. It allows the arcade to sell from basic use cases to thousands of daily transitions while preserving a completely accurate historical record of customer activity.

Example Queries

See spending per game

Check to see what machines have the highest token spends. We left join play sessions since some machines could be unplayed and we would want to display a zero. play_session inner joins play_payment since this is a 1:1 relationship.

```
SELECT machine_name, SUM(tokens_spent) AS total_tokens
FROM machine LEFT JOIN play_session USING (machine_id)
INNER JOIN play_payment USING (session_id)
GROUP BY machine_name
ORDER BY SUM(tokens_spent) DESC;
```

See spending per customer

Find the whales! Who are the top spenders at the arcade? We can inner join because we only want players with positive spending. We don't care if this is across more than one card. This includes inactive cards since they may have had spending in the past.

```
SELECT customer_id, first_name, last_name, SUM(tokens_spent) AS
total_tokens
FROM customer INNER JOIN card USING (customer_id)
INNER JOIN play_session USING (card_id)
INNER JOIN play_payment USING (session_id)
GROUP BY customer_id, first_name, last_name
ORDER BY SUM(tokens_spent) DESC LIMIT 10;
```

Ticket redemption per user

Who spends the most tickets on prizes? This query calculates the total number of tickets each customer has spent on redemptions, including customers who have never redeemed and therefore show 0. This includes inactive cards since they may have had redemptions in the past.

```
SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  COALESCE(SUM(ri.quantity * ri.tickets_cost_each), 0) AS
tickets_spent
FROM customer c
LEFT JOIN card USING (customer_id)
LEFT JOIN redemption USING (card_id)
LEFT JOIN redemption_item USING (redemption_id)
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY tickets_spent DESC;
```

High scoring players per machine

Who are the highest scoring players on each machine? This helps us see top performers and identify competitive players. This can be used in the context of a leaderboard showcasing the top performing players per machine. This also sorts data from top performers first to least performing players. The score range is limited to higher scoring players to eliminate noise from games with only poor performances.

```
SELECT
  m.machine_name,
  c.customer_id,
  c.first_name,
  c.last_name,
  ps.score
FROM machine m
INNER JOIN play_session ps USING (machine_id)
INNER JOIN card ca USING (card_id)
INNER JOIN customer c USING (customer_id)
WHERE ps.score BETWEEN 500 AND 9999
ORDER BY m.machine_name, ps.score DESC;
```

Top Machines by Average Ticket Payout

Management wants to analyze ticket distribution and identify which machines are the most generous. This query computes the average tickets awarded per play and filters out machines with low activity to prevent small sample sizes from distorting the rankings.

```
SELECT m.machine_id, m.machine_name, m.machine_type,
ROUND(AVG(tickets_awarded), 2) AS avg_tickets_per_play
FROM machine AS m
INNER JOIN play_session USING (machine_id)
INNER JOIN ticket_award USING (session_id)
INNER JOIN ticket_rule USING (rule_id)
GROUP BY m.machine_id, m.machine_name, m.machine_type
HAVING COUNT(*) > 10
ORDER BY avg_tickets_per_play DESC
LIMIT 5;
```

Active Players Without Prize Redemptions

Management wants to identify active customers who are engaging with specific machine types but have not yet redeemed any prizes. This query uses DISTINCT to avoid duplicate customers, IN to filter targeted machine types, NOT IN to exclude inactive cards, and a LEFT JOIN with IS NULL to detect customers who have no redemption records.

```
SELECT DISTINCT c.customer_id, c.first_name, c.last_name
FROM customer AS c
JOIN card AS ca USING (customer_id)
JOIN play_session AS ps USING (card_id)
JOIN machine AS m USING (machine_id)
LEFT JOIN redemption AS r USING (card_id)
WHERE ca.card_status NOT IN ('inactive') AND r.redemption_id IS
NULL AND m.machine_type IN ('racing', 'rhythm', 'basketball')
ORDER BY c.customer_id;
```

Query Optimization

1. Adding an index on `purchase(card_id, purchased_at)` would help with showing a time-ordered purchase history per card.
2. Adding an index on `ticket_rule(machine_id,active,min_score,max_score)` would help with finding matching active rules to issue tickets based on a score.